

从 LeetCode 1219 与 3459 看：什么时候 DP 成立？

在刷题过程中，我们经常会遇到这样一种困惑：

明明感觉问题很“动态规划”，但就是不知道状态怎么定义、表怎么填；而另一些看起来同样是路径问题的题，却可以非常自然地用 DP 解决。

LeetCode **1219《黄金矿工》** 与 **3459《最大 V 型对角线》**，正好构成了一组极具代表性的对比样本。

本文尝试通过对这两道题的对照分析，讨论一个核心问题：

一个问题“能不能 DP”，本质上取决于什么？

一、问题简述

1. LeetCode 1219：黄金矿工

- 给定一个网格，每个格子有一定数量的黄金（或为 0）
- 可以从任意非 0 格子出发
- 每个格子最多访问一次
- 上下左右移动
- 目标：收集尽可能多的黄金

这是一个典型的「网格路径最大化」问题。

2. LeetCode 3459：最大 V 型对角线

- 给定一个矩阵
- 寻找形如 V 字的对角线路径
- 路径方向固定
- 在某个拐点处由一条对角线切换到另一条对角线
- 目标：最大化路径权值

同样是路径问题，但官方/主流解法是 **动态规划**。

二、为什么 1219 很难（也不该）用 DP？

1. 必须面对的状态定义

如果我们尝试为 1219 定义 DP 状态，会很快遇到一个现实问题：

$dp[x][y]$ = 从 (x, y) 出发能获得的最大黄金？

这个定义是**不成立的**。

原因在于：

从同一个 (x, y) 出发，未来能走哪些格子，完全取决于“之前已经走过哪些格子”。

因此，完整状态至少应为：

$(x, y, \text{visited})$

其中 **visited** 是一个集合（或位掩码），表示已经访问过的格子。

2. visited 带来的根本问题

一旦状态中包含 **visited**：

- 状态数量呈指数级增长
- 不同路径顺序会生成完全不同的状态
- **不存在一个统一的计算顺序**

更重要的是：

无法找到一个顺序，使得所有状态只依赖于“已经计算过”的状态。

也就是说：

- 状态依赖图虽然是 DAG（mask 的子集关系）
- 但**无法线性化**

这正是 DP 填表“无从下手”的根本原因。

3. 本质判断

1219 的问题本质是：

在网格图中，寻找一条 **权值最大的简单路径（simple path）**。

而“简单路径 + 任意拐弯 + 不可重复访问”，天然就是：

搜索问题，而不是状态累积问题。

因此，DFS + 回溯并不是“退而求其次”，而是**最符合问题结构的解法**。

三、为什么 3459 可以自然地 DP？

与 1219 形成鲜明对比，3459 具备 DP 的一系列“理想条件”。

1. 路径方向固定

V 型路径可以拆分为两段：

- 一段沿固定对角线方向前进
- 另一段沿另一固定对角线方向前进

例如：

```
(i, j) 只依赖于 (i-1, j-1)
(i, j) 只依赖于 (i-1, j+1)
```

这立即带来了一个关键性质：

存在天然的先后顺序（拓扑序）。

2. 子问题含义稳定

在 3459 中，一个常见状态是：

```
dp[i][j][dir]
```

其含义是：

从某个固定方向走到 (i, j) 时，所能获得的最大值。

注意这里的两个特点：

- 状态只与「位置 + 方向」有关
- **与具体走过哪些点无关**

因此：

同一个状态，在任何路径下含义都是一致的。

这是 DP 成立的一个关键前提。

3. 子问题可独立累积

V 型路径并不是“选择一条完整路径”，而是：

- 每个点都可以独立计算“从某方向到达这里的最优值”
- 在拐点处进行合并

这是一种典型的 **状态最优值累积模型**，而不是路径搜索模型。

四、抽象总结：什么时候 DP 成立？

通过这两个问题的对比，可以提炼出一个非常重要的判断标准：

如果一个问题的状态依赖关系可以被线性化，使得每个状态只依赖于“已计算过”的状态，那么 DP 就成立。

换一种更图论的说法：

状态依赖图不仅要是 DAG，还必须“容易被线性化”。

五、一个实用的对照表

维度	LeetCode 1219	LeetCode 3459
是否需要 visited	是	否
路径是否自由	是	否
状态是否稳定	否	是
是否存在全序	否	是
本质	路径搜索	状态累积
合理解法	DFS / 回溯	DP

六、结语

当我们发现一个问题：

- 状态中必须包含“已经走过哪些点”
- 下一步选择强烈依赖完整历史路径

那么，与其强行 DP，不如承认它的本质：

这是一个搜索问题。

反之，只要我们能：

- 为状态建立稳定含义
- 找到明确的计算顺序

那么 DP 往往会水到渠成。

DP 并不是万能工具，它成立的前提，本身就是问题结构给出的。